

Task 4-c

Section 1: Motion control plugins

1.1 Diff Drive Plugin

A. Plugin identity

<plugin

Filename="libignition-gazebo-diff-drive-system.so"

Name="ignition::gazebo::systems::DiffDrive">

</plugin>

B.Functional Description

From the name itself as it says diff drive it means there are two independent wheels, these are present on either side of the robot. It tracks the vehicle's motion and manages wheel's motion according to it.

Here the plugin subscribes to a velocity command topic and calculates the necessary target rotational velocities for the right and left wheel joints based on robot's physical dimension and updates joint states natively via physics engine. It also computes and publishes odometry data based on forward kinematics.

It should be used whenever we are simulating a differential drive robot and want to control it directly using speed vectors without writing a custom joint controller.

C.Key parameter table

<left_joint> (string data type) (it is the name of the joint connecting the chasis to the left wheels)

<right_joint>(string data type) (it is the name of the joint connecting the chasis to the right wheels)

<wheel_separation>(double data type)(The lateral distance between the centerlines of the left and right wheels (in meters))

<wheel_radius> (double data type)(The operational radius of the driving wheels (in meters))

<update_rate>(double data type)(The frequency (in Hz) at which the odometry is calculated and published)

<topic>(string data type)(The target input topic name for incoming velocity commands (Defaults to /cmd_vel))

D.Sample xml snippet

<gazebo>

<plugin

filename="libignition-gazebo-diff-drive-system.so"

name="ignition::gazebo::systems::DiffDrive">

<left_joint>left_wheel_joint</left_joint>

<right_joint>right_wheel_joint</right_joint>

<wheel_separation>0.45</wheel_separation>

<wheel_radius>0.15</wheel_radius>

<update_rate>50.0</update_rate>

<topic>/cmd_vel</topic>

<odom_topic>/odom</odom_topic>

<frame_id>odom</frame_id>

<child_frame_id>base_footprint</child_frame_id>

</plugin>

</gazebo>

E.Ros2 bridge Mapping

To pass message between ROS 2 Humble and Ignition Gazebo Fortress, w must use `ros_gz_bridge` (or `ros_ign_bridge`).The configurations for both control commands and odometry data feedback.

Command Velocity (`cmd_vel`)

- ROS 2 Message Type: `geometry_msgs/msg/Twist`
- Ignition Message Type: `ignition.msgs.Twist`

Bridge configuration string:

```
ros2 run ros_gz_bridge parameter_bridge  
/cmd\_vel@geometry\_msgs/msg/Twist|ignition.msgs.Twist
```

Odometry Feedback (`odom`)

- ROS 2 Message Type: `nav_msgs/msg/Odometry`
- Ignition Message Type: `ignition.msgs.Odometry`

Bridge configuration string:

```
ros2 run ros_gz_bridge parameter_bridge  
/odom@nav\_msgs/msg/Odometry|ignition.msgs.Odometry
```

1.2 Joint controller plugin

A. Plugin identity

<plugin

filename="libignition-gazebo-joint-controller-system.so"

name="ignition::gazebo::systems::JointController">

</plugin>

B.Functional Description

It allows direct operational control over a single designated joint (specifically its first axis) within a simulated robot model

It listens for incoming control commands over a chosen topic. By default, it operates in Velocity Mode, tracking target joint velocities directly. If configured for force actuation (<use_force_commands> set to true), it runs a closed-loop Proportional-Integral-Derivative (PID) loop internally to compute the force/torque commands needed to hit target inputs

It should be used when we need simple, decoupled control over individual rotating or translating parts—such as a robotic arm joint, a conveyor wheel, a steering pivot, or an isolated sensor tilt axis—without adding a heavy multi-joint trajectory controller framework.

C.Key parameter table

<joint_name>(string data type)(The specific target joint name within the model to actuate)

<topic>(string data type)(Custom topic name to receive scalar commands)

<initial_velocity>(double data type)(Starting velocity applied to the joint at simulation ignition (in rad/s or m/s))

<use_force_commands>(Boolean data type)(Set to true to implement the controller using force/torque via a PID loop. Defaults to false (Velocity Mode))

<p_gain>(double data type)(Proportional gain for the PID loop (Used exclusively in force mode; Defaults to 1))

<i_gain>(double data type)(Integral gain for the PID loop (Used exclusively in force mode; Defaults to 0))

<d_gain>(double data type)(Derivative gain for the PID loop (Used exclusively in force mode; Defaults to 0))

D.Sample xml snippet

```
<gazebo>
```

```
  <plugin
```

```
    filename="libignition-gazebo-joint-controller-system.so"
```

```
    name="ignition::gazebo::systems::JointController">
```

<joint_name>arm_elbow_joint</joint_name>

<topic>/arm/elbow/cmd_vel</topic>

<initial_velocity>0.5</initial_velocity>

<use_force_commands>>false</use_force_commands>

</plugin>

</gazebo>

E.Ros2 bridge Mapping

To communicate with this plugin using a ROS 2 stack (humble), we must bridge the transport topics.

Joint Velocity Command (cmd_vel)

- ROS 2 Message Type: std_msgs/msg/Float64 (or std_msgs/msg/Double depending on setup)
- Ignition Message Type: ignition.msgs.Double

Bridge configuration string:

ros2 run ros_gz_bridge parameter_bridge

[/arm/elbow/cmd_vel@std_msgs/msg/Float64|ignition.msgs.Double](#)

Section 2: Sensor plugins

2.1 IMU Sensor plugin

A. Plugin identity

<plugin

```
filename="libignition-gazebo-imu-system.so"
name="ignition::gazebo::systems::Imu">
</plugin>
```

B.Functional Description

It simulates an inertial measurement unit attached to a robot link, generating physical state telemetry relative to a fixed inertial space.

It monitors the host physics frame to calculate real-time rotational orientations (represented as Quaternions), 3-axis angular velocity vectors (X, Y, Z), and 3-axis linear acceleration components (X, Y, Z) experiencing simulated gravitation effects. This sensor state is then processed internally and dispatched to downstream subscriber channels at configured operational loops

It should be used whenever a model relies on precise orientation data, dead-reckoning state estimations, or specialized feedback controllers (such as self-balancing platforms, local odometry localization tools like `robot_localization`, and quadcopter orientation tracking).

C.Key parameter table

`name="..."`(attribute)(string data type)(it gives unique reference identify tag assigned to the specific IMU instance)

`type="imu"`(attribute)(string data type)(Tells the simulator to process the block as an Inertial Measurement Unit)

`<always_on>`(Boolean data type)(Set to 1 (true) to ensure the sensor thread updates continuously without external request triggers)

`<update_rate>`(double data type)(The specific output generation rate frequency (expressed in Hz))

`<visualize>`(Boolean data type)(Set to true to render visual debugging telemetry markers inside the Ignition simulation GUI environment)

`<topic>`(string data type)(Defines the target topic name where structured IMU telemetry data is actively published)

`<gz_frame_id>`(string data type)(Sets the output coordinate frame tracking metadata string reference for published IMU datasets)

D.Sample xml snippet

```
<plugin
  filename="libignition-gazebo-imu-system.so"
  name="ignition::gazebo::systems::Imu">
</plugin>
```

```
<link name="chassis">
  <sensor name="robot_imu" type="imu">
    <always_on>1</always_on>
    <update_rate>100.0</update_rate>
    <visualize>true</visualize>
    <topic>/imu/data</topic>
    <gz_frame_id>imu_link</gz_frame_id>
  </sensor>
</link>
```

E.Ros2 bridge Mapping

IMU Telemetry Data (/imu/data)

- ROS 2 Message Type: `sensor_msgs/msg/Imu`
- Ignition Message Type: `ignition.msgs.IMU`

Bridge configuration string:

```
ros2 run ros_gz_bridge parameter_bridge
/imu/data@sensor\_msgs/msg/Imu\[ignition.msgs.IMU
```

2.2 GPS Sensor plugin

A. Plugin identity

```
<plugin  
  filename="libignition-gazebo-navsat-system.so"  
  name="ignition::gazebo::systems::NavSat">  
</plugin>
```

B.Functional Description

It (internally referenced as NavSat) simulates a satellite navigation receiver attached to a link, tracking coordinate data mapped to geographic locations on Earth.

The plugin reads the model's absolute position inside the simulation world frame and cross-references it with the spherical coordinates (latitude, longitude, elevation) configured in the world file. It computes real-time positional updates along with optional noise profiles and publishes the telemetry on an outgoing simulation channel

It is to be used when simulating outdoor mobile robots, autonomous drones, or marine craft where global navigation state estimation, waypoint tracking, or outdoor multi-sensor fusion (using packages like robot_localization) is required

C.Key parameter table

name="..."(attribute)(string data type)(Unique reference identification tag assigned to the specific GPS instance)

type="navsat"(string data type)(Tells the simulator to process the block as a Navigation Satellite (GPS) sensor)

<always_on>(Boolean data type)(Set to 1 (true) to ensure the sensor thread updates continuously without external request triggers)

<update_rate>(double data type)(The specific data generation output rate frequency (expressed in Hz))

<topic>(string data type)(Defines the target topic name where structured NavSat telemetry data is actively published (Defaults to /navsat))

<gz_frame_id>(string data type)(Sets the output coordinate frame tracking metadata string reference for published navigation datasets)

D.Sample xml snippet

<plugin

filename="libignition-gazebo-navsat-system.so"

name="ignition::gazebo::systems::NavSat">

</plugin>

<link name="gps_link">

<sensor name="robot_gps" type="navsat">

<always_on>1</always_on>

<update_rate>10.0</update_rate>

<topic>/gps/data</topic>

<gz_frame_id>gps_link</gz_frame_id>

</sensor>

</link>

E.Ros2 bridge Mapping

To share global navigation state telemetry across an active ROS 2 pipeline (Foxy/Humble), we need to deploy `ros_gz_bridge`

GPS Global Position Data (/gps/data)

- ROS 2 Message Type: `sensor_msgs/msg/NavSatFix`
- Ignition Message Type: `ignition.msgs.NavSat`

Bridge configuration string:

`ros2 run ros_gz_bridge parameter_bridge`

[/gps/data@sensor_msgs/msg/NavSatFix\[ignition.msgs.NavSat](#)

2.3 Magnetometer Sensor plugin

A. Plugin identity

```
<plugin  
  filename="libignition-gazebo-magnetometer-system.so"  
  name="ignition::gazebo::systems::Magnetometer">  
</plugin>
```

B.Functional Description

It simulates a digital compass or magnetic flux sensor, tracking the local ambient magnetic field vectors relative to the robot's physical frame

The plugin measures the magnetic field vector surrounding the sensor body. It cross-references the robot link's position and orientation against the world's ambient magnetic field vector (which can be configured inside the world SDF file). It can also factor in noise profiles to accurately mimic hardware distortion before publishing the 3-axis (\$X, Y, Z\$) tesla data on a simulation channel

It has to be used when simulating navigation stacks that depend on a digital compass heading, heading correction for outdoor planetary/terrestrial rovers, or multi-sensor fusion algorithms (like robot_localization) that combine IMU and magnetic headings to minimize long-term yaw drift

C.Key parameter table

name="..."(attribute)(string data type)(Unique reference identification tag assigned to the specific Magnetometer instance.)

type="magnetometer" (Attribute)(Tells the simulator to process the block as a Magnetometer sensor)

<always_on>(Boolean data type)(Set to 1 (true) to ensure the sensor thread updates continuously without external request triggers)

<update_rate>(double data type)(The specific data generation output rate frequency (expressed in Hz))

<topic>(string data type)(Defines the target topic name where structured magnetic field telemetry data is actively published (Defaults to /magnetometer))

<gz_frame_id>(string data type)(Sets the output coordinate frame tracking metadata string reference for published magnetometer datasets)

D.Sample xml snippet

<plugin

filename="libignition-gazebo-magnetometer-system.so"

name="ignition::gazebo::systems::Magnetometer">

</plugin>

<link name="mag_link">

<sensor name="robot_mag" type="magnetometer">

<always_on>1</always_on>

<update_rate>50.0</update_rate>

<topic>/mag/data</topic>

<gz_frame_id>mag_link</gz_frame_id>

</sensor>

</link>

E.Ros2 bridge Mapping

To route magnetic field strength data across an active ROS 2 pipeline (Foxy/Humble), we need to deploy ros_gz_bridge

Magnetic Field Data (/mag/data)

- **ROS 2 Message Type: sensor_msgs/msg/MagneticField**
- **Ignition Message Type: ignition.msgs.Magnetometer**

Bridge configuration string:

```
ros2 run ros_gz_bridge parameter_bridge
```

```
/mag/data@sensor\_msgs/msg/MagneticField\[ignition.msgs.Magnetometer
```

2.4 Contact Sensor plugin

A. Plugin identity

```
<plugin
```

```
  filename="libignition-gazebo-contact-system.so"
```

```
  name="ignition::gazebo::systems::Contact">
```

```
</plugin>
```

B.Functional Description

It simulates a bumper or pressure-sensitive boundary, detecting mechanical collisions between a robot link's collision geometries and external obstacles.

It monitors designated structural collision shapes managed by the physics engine. When physical contact occurs, it collects contact points, surface normals, penetration depths, and resultant interaction forces (X,Y,Z). This spatial contact block is then packaged and dispatched over simulation transport channels

It should be used when implementing bumper switches for vacuum robots, landing-gear impact detectors for drones, protective collision stops for industrial manipulators, or feedback networks requiring immediate notification when a structural shell strikes an obstacle

C.Key parameter table

name="..." (Attribute)(string data type)(Unique reference identification tag assigned to the specific Contact instance)

type="contact" (Attribute)(container data type)(Scoping tag enclosing the specific element links intended to monitor contact states)

<collision>(string data type)(The exact name string matching the specific target <collision> geometry tag within the parent link)

<always_on>(Boolean data type)(Set to 1 (true) to ensure the sensor thread updates continuously without external request triggers)

<update_rate>(double data type)(The specific data generation output rate frequency (expressed in Hz))

<topic>(string data type)(it defines the target topic name where contact states and force vectors are published (Defaults to /sensor_contact))

D.Sample xml snippet

<plugin

filename="libignition-gazebo-contact-system.so"

name="ignition::gazebo::systems::Contact">

</plugin>

<link name="bumper_link">

<collision name="bumper_collision">

<geometry>

<box>

<size>0.1 0.5 0.1</size>

</box>

</geometry>

</collision>

<sensor name="robot_bumper" type="contact">

<always_on>1</always_on>

<update_rate>50.0</update_rate>

<topic>/bumper/data</topic>

<contact>

```
<collision>bumper_collision</collision>
</contact>
</sensor>
</link>
```

E.Ros2 bridge Mapping

To route collision feedback structures across an active ROS 2 pipeline (Foxy/Humble), we need to deploy `ros_gz_bridge`

Contact / Bumper State Data (`/bumper/data`)

- ROS 2 Message Type: `ros_gz_interfaces/msg/Contact`
- Ignition Message Type: `ignition.msgs.Contacts`

Bridge configuration string:

```
ros2 run ros_gz_bridge parameter_bridge
/bumper/data@ros_gz_interfaces/msg/Contact[ignition.msgs.Contacts
```

2.5 Camera Sensor plugin

A. Plugin identity

```
<plugin
  filename="libignition-gazebo-sensors-system.so"
  name="ignition::gazebo::systems::Sensors">
</plugin>
```

B.Functional Description

It simulates a standard monocular optical camera attached to a robot frame, capturing and streaming live 2D image matrices from the simulated environment

It interfaces with the Ignition Rendering engine to generate a visual frame from the perspective of the sensor's origin. It processes scene properties, lighting, and textures, then outputs raw pixel matrices (such as RGB images)

alongside corresponding optical camera metadata matrices
(sensor_msgs/msg/CameraInfo)

It is to be used whenever your robot relies on visual data streams—such as visual servoing, color-based object detection, lane tracking, deep learning model training, or streaming a perspective viewport to an operator dashboard interface

C.Key parameter table

name="..." (Attribute)(string data type)(Unique reference identification tag assigned to the specific Camera instance)

type="camera" (Attribute)(string data type)(it tells the simulator to process the block as an optical camera system)

<always_on>(Boolean data type)(Set to 1 (true) to ensure the sensor thread updates continuously without external request triggers)

<update_rate>(double data type)(The image frame generation output rate frequency (expressed in frames per second, or Hz))

<visualize>(Boolean data type)(Set to true to render a visual preview frustum frame inside the Ignition simulation GUI environment)

<topic>(string data type)(it defines the target base topic name where structured raw image data streams are published (Defaults to /camera))

<camera>(container data type)(Scoping container tag enclosing the operational characteristics of the lens array)

<horizontal_fov>(double data type)(The horizontal field of view of the simulated lens (expressed in radians))

<image>(container data type)(it is req as it encloses image matrix resolution values and pixel encoding schemas (<width>, <height>, <format>))

D.Sample xml snippet

```
<plugin
```

```
  filename="libignition-gazebo-sensors-system.so"
```

```
  name="ignition::gazebo::systems::Sensors">
```

```
</plugin>
```

```
<link name="camera_link">

  <sensor name="front_camera" type="camera">

    <always_on>1</always_on>

    <update_rate>30.0</update_rate>

    <visualize>true</visualize>

    <topic>/camera/image_raw</topic>


  <camera>

    <horizontal_fov>1.3962634</horizontal_fov>

    <image>

      <width>640</width>

      <height>480</height>

      <format>R8G8B8</format>

    </image>

    <clip>

      <near>0.1</near>

      <far>100.0</far>

    </clip>

  </camera>

</sensor>

</link>
```

E.Ros2 bridge Mapping

To route raw pixel matrices across an active ROS 2 pipeline (Foxy/Humble), we need to deploy `ros_gz_bridge`

Camera Image Stream (/camera/image_raw)

- **ROS 2 Message Type: sensor_msgs/msg/Image**
- **Ignition Message Type: ignition.msgs.Image**

Bridge configuration string:

ros2 run ros_gz_bridge parameter_bridge

[/camera/image_raw@sensor_msgs/msg/Image\[ignition.msgs.Image](#)

